

Ejemplo de Estructuras

Las estructuras (palabra clave: Struct) son tipos de datos ordenados y estructurados parecidos a las clases, pueden tener constructores, funciones miembro, y por supuesto miembros de tipos de datos mas primitivos e incluso otro struct.

Tambien se pueden definir como definición de tipo (typedef) que permite crear varios objetos con la misma estructura, herencia y polimorfismo.

Las estructuras y las clases en C++ permiten casi lo mismo, siendo la única diferencia: el nivel de protección por defecto.

Tipo de Derivada	Sintaxis	Protección
struct	struct B : A {};	Pública (los miembros públicos de A siguen siendo públicos en B)
class	class B : A {};	Privada (los miembros públicos de A se vuelven inaccesibles (Privados) en B)

Introduccion a la Estructura

```
1  /*****
2  Instituto Tecnológico de Durango
3  Departamento Metal-Mecanica
4
5  Programacion Avanzada 7U
6  Fabian Quiñones Escarzaga 23040871
7  Ing. Mario Gerardo Hernandez Marines
8  Clase Estructuras
9  *****/
10 #include <iostream>
11 #include <string.h>
12
13 using namespace std;
14
15 struct {
16     int age;
17     char CountryCode[3];
18     char Name[256];
19     void print(){ // Véase como la struct puede tener funciones como miembros.
20         cout << "Nombre: " << Name << " Pais: " << CountryCode << " Edad: " << age;
21     };
22 } Luis; // Observe como pongo el nombre del OBJETO abajo y despues de la
        declaracion del struct antes del punto y coma para definir un objeto, si se
        colocara después de struct fuera un tipo de dato organizado;
23
24 int main(){
25     Luis.age =20;
26     strcpy(Luis.CountryCode,"MX");
27     strcpy(Luis.Name,"Luis Perez");
28
29     Luis.print();
30
31     return 0;
32 }
```

Registro de Ejecución (Interacción)

Nombre: Luis Perez Pais: MX Edad: 20

Código Fuente

```
1  /*****
2  Instituto Tecnológico de Durango
3  Departamento Metal-Mecanica
4
5  Programacion Avanzada 7U
6  Fabian Quiñones Escarzaga 23040871
7  Ing. Mario Gerardo Hernandez Marines
8  Clase Estructuras Herencia
9  *****/
10 #include <iostream>
11 #include <string.h>
12
13 using namespace std;
14
15 struct persona {
16     int age;
17     char CountryCode[3];
18     char Name[256];
19     virtual void print() { // Véase como la struct puede tener funciones como
        miembros.
20         cout << "Nombre: " << Name << " Pais: " << CountryCode << " Edad: " << age;
21     };
22 };
23
24 struct trabajador : persona { // Véase que el trabajador hereda de persona age,
    CountryCode, Name los cuales son miembros públicamente accesibles
25     int horas_a_la_semana;
26     int nomina_semanal;
27     // Observe como se realiza un "override" o invalidación de la función de la
        clase base para implementar la propia de la hija, se necesita que la funció
        n sea virtual void
28     void print() override{
29         cout << "Nombre: " << Name << " | Pais: " << CountryCode << " | Edad: " <<
            age << " | Horas a la semana: " << horas_a_la_semana << " | Nomina
            Semanal " << nomina_semanal;
30     };
31 };
32
33 int main(){
34     trabajador Agustin;
35     Agustin.horas_a_la_semana = 40;
36     Agustin.nomina_semanal = 8000;
37     //Sin mayor preambulo puedo acceder directamente a los miembros de persona del
        trabajador
38     Agustin.age =20;
39     strcpy(Agustin.CountryCode,"MX");
40     strcpy(Agustin.Name,"Agustin Perez");
41
42     Agustin.print();
43     return 0;
44 }
```

Registro de Ejecución (Interacción)

Nombre: Agustin Perez Pais: MX Edad: 20 Horas a la semana: 40 Nomina Semanal 8000
--

Si en la línea 19 se escribiera "struct trabajador : private persona {" no se pudiera acceder a los miembros públicos de persona y las líneas 30-34 serían incorrectas.